

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 2 – AI Food Analyzer

Team:	StomachSlave	Date:	May 24, 2026
Repo:	https://github.com/GeminusF/ai-eng-110-final-project-ss-stomachslave	Tag:	v1.0-final created
Members:	Fereh Feyzullayev (@GeminusF), Milana Karimova (@milanakarimova), Nicat Agayev (@Nicat-Agayev)		

Executive Summary

StomachSlave is a Topic 2 AI Food Analyzer that accepts a meal image, identifies visible ingredients and estimated portions, retrieves nutrition facts, computes calories and macronutrients, and exposes the same workflow through a Typer CLI, FastAPI HTTP API, and HTMX web UI. The implementation keeps the provided `ai/` package as a boundary and wraps it with configuration, retries, caching, logging, validation, storage, and bounded asynchronous orchestration. The benchmark command `python scripts\bench.py -image data\rice_chicken_broccoli.png` measured 3 offline nutrition lookups with sequential time 0.0013s and concurrent time 0.0013s; this does not show a wall-clock speedup because the fake providers return almost immediately, but the concurrency behavior is directly tested with a semaphore-bound workload. The robustness story is that invalid images are rejected before provider calls, unknown meals return a structured `unknown_meal` result, and individual nutrition lookup failures become warnings so the pipeline can still return partial results when possible. The full test suite passed in the project virtual environment with 88 tests passing and 85.62% coverage, including the required 29 provided AI smoke tests. If we restarted the project, the main change would be to build a benchmark with realistic artificial latency so the performance section better demonstrates the value of parallel USDA calls.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing and scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design and key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency and Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit and politeness	5

4	Robustness and Error Handling	5
4.1	Wrapping the AI module	5
4.2	Failure-mode analysis	5
4.3	Input validation	6
5	Storage and Caching	6
5.1	Schema	6
5.2	Caching policy	6
6	Testing	7
6.1	Strategy and coverage	7
6.2	Notable tests	7
7	Deployment and Reproducibility	7
7.1	Docker image	7
7.2	Configuration	8
8	Limitations and Future Work	8
9	Tools and Acknowledgements	8
	References	9
A	Appendix – Reproducing the benchmark	9

1. Project Overview

1.1 Problem framing and scope

The problem from `TOPIC.md` is to build the software-engineering layer around a provided AI module for meal-image analysis. The user supplies a PNG or JPEG meal image. The system identifies visible ingredients with estimated weights, looks up nutrition facts for each ingredient, and returns per-ingredient and total calories, protein, carbohydrates, and fat.

The project has three user-facing entry points. The CLI supports `python -m foodanalyzer analyze <path>`, optional `-offline`, optional `-json`, and a `history` command. The HTTP API exposes `GET /health`, `POST /analyze`, `GET /analyses`, and `GET /analyses/{analysis_id}`. The HTMX web UI is mounted under `/ui` and uses the same analyzer service as the API and CLI.

The reproducible demo path uses local fake providers in `foodanalyzer.offline`, so it needs no API keys or network access. The online path supports OpenRouter as an app-layer VLM adapter and USDA FoodData Central as the nutrition provider, with PostgreSQL used for analysis history and the persistent nutrition cache. There is no embedding provider in Topic 2 because this topic does not require embeddings.

1.2 Provider choice and the AI module contract

The original provided AI package supports Anthropic, OpenAI, and Google Gemini VLM providers through `LLM_PROVIDER`. This project also adds `OpenRouterVLM` in the SE layer, configured with `LLM_PROVIDER=openrouter`, `LLM_MODEL`, `OPENROUTER_API_KEY`, and `OPENROUTER_BASE_URL`. The default example model in `.env.example` is `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`. Nutrition lookup uses the provided `ai.NutritionProvider` contract, with the online app path using `ai.USDAProvider`.

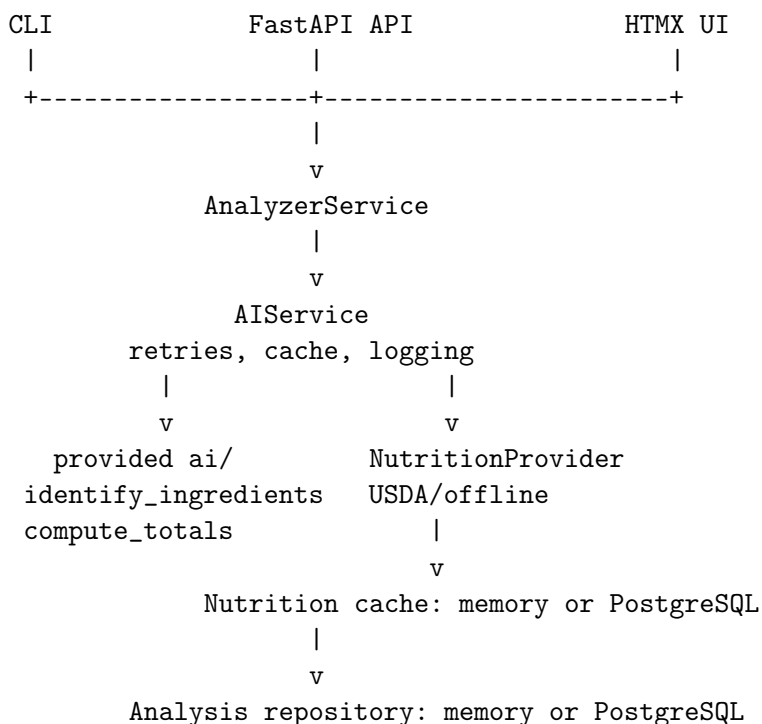
The application calls the provided AI surface through `ai.identify_ingredients`, `ai.compute_tot`

als, and the `ai.NutritionProvider.lookup` method. We did not modify the public interface of the provided `ai/` package. The OpenRouter integration implements the existing `VLMProvider` interface and is injected into `ai.identify_ingredients(..., vlm=provider)` rather than bypassing the course package.

Malformed model responses are handled in two layers. The provided `ai.identify_ingredients` validates JSON against its schema and pydantic models reject negative grams, invalid confidence, missing fields, empty names, and extra fields. The OpenRouter adapter also strips Markdown fences, extracts JSON candidates, retries with a stricter JSON-only prompt, and repairs limited cases such as a missing top-level `meal_recognized` flag when ingredients are present.

2. Architecture

2.1 Module map



The provided `ai/` package boundary is crossed only from `AIService` and the OpenRouter adapter that implements the provided `VLMProvider` interface. Storage is hidden behind repository/cache protocols, so the CLI, API, and web UI do not know whether the backing implementation is in-memory or PostgreSQL. Swapping from OpenRouter to one of the built-in providers mostly changes environment variables and the VLM construction in `foodanalyzer.cli.build_vlm`; the analyzer and routes remain unchanged.

`foodanalyzer.api`

Creates the FastAPI app, validates multipart uploads, saves safe upload filenames, and returns pydantic response models.

`foodanalyzer.cli`

Builds configured services and provides the table/JSON command-line interface.

`AnalyzerService`

Owns the business workflow: identify ingredients, handle unknown meals, fetch nutrition in parallel, compute totals, and save records.

`AIService`

Wraps the provided AI calls with retries, logging, and cache-aware nutrition lookup.

concurrency.pipeline

Runs bounded parallel nutrition lookups and converts individual provider failures into warnings.

storage.repository

Persists analysis history through either an in-memory repository or PostgreSQL.

services.nutrition_cache

Provides TTL cache implementations for memory and PostgreSQL.

2.2 OOP design and key abstractions

The project uses composition around protocols for storage and caching. This is a good fit because the analyzer only needs a small behavioral contract; it should not care whether results are stored in memory during offline tests or in PostgreSQL during online runs.

```

1 class AnalysisRepository(Protocol):
2     async def save(self, result: AnalysisResult) -> AnalysisRecord: ...
3     async def list_recent(self, limit: int = 10) -> list[AnalysisRecord]: ...
4     async def get(self, analysis_id: str) -> AnalysisRecord | None: ...
5     async def health(self) -> bool: ...
6
7 class InMemoryAnalysisRepository:
8     ...
9
10 class PostgresAnalysisRepository:
11     ...

```

The analyzer receives an `AnalysisRepository` through its constructor, which is composition rather than inheritance. That makes the same core workflow usable in tests, offline demos, API routes, CLI commands, and PostgreSQL-backed online mode. The cache uses the same pattern through `NutritionCacheBackend`; `OpenRouterVLM` is the inheritance example because it implements the provided abstract `VLMProvider`.

2.3 Data flow across module boundaries

The SE-layer models are `AnalysisStatus`, `NutritionTotals`, `IngredientResult`, `AnalysisResult`, `AnalysisRecord`, and `ErrorResponse`. These are pydantic models with forbidden extra fields where appropriate, so API responses, repository payloads, and analyzer outputs are typed. The code also uses provided `ai.Ingredient`, `ai.NutritionFacts`, and `ai.Nutrition` values at the AI boundary.

The project does use dictionaries internally for provider payloads, cache maps, and `facts_by_name`. User-facing and storage-facing boundaries use pydantic models, and the main analyzer contract returns `AnalysisResult` or `AnalysisRecord`, not an unstructured dictionary.

3. Concurrency and Performance

3.1 Why this concurrency model

Nutrition lookups are independent after the VLM returns a list of ingredients, so the project uses `asyncio.gather` with `asyncio.Semaphore`. The default bound is `MAX_CONCURRENCY=10`, configured through pydantic settings. The target bottleneck is I/O latency from USDA/provider calls rather than CPU work, so process-based parallelism would add overhead without helping.

The current implementation wraps blocking provider calls with `asyncio.to_thread`, then coordinates them from async code. Setting the semaphore to 1 makes the nutrition stage effectively

sequential. Setting it very high would increase burst pressure on providers and PostgreSQL; the implementation defaults to 10 as a conservative request-level bound.

3.2 Sequential-vs-concurrent benchmark

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Offline lookup for <code>rice_chicken_broccoli.png</code>	3	0.0013	0.0013	1.00x

The benchmark was run on Windows with Python 3.12.10 from the project virtual environment using:

```
.venv\Scripts\python.exe scripts\bench.py --image data\rice_chicken_broccoli.png
```

The cache is cleared between the sequential and concurrent portions inside `scripts/bench.py`. The result is honest but not very illustrative: the offline fake provider is so fast that scheduling overhead dominates and there is no measurable speedup. The concurrency design is still verified by `tests/test_pipeline.py::test_pipeline_bounds_parallelism`, which creates five async lookup tasks and asserts that no more than the configured limit are active at once.

3.3 Rate limit and politeness

The project does not implement a token bucket or request-per-minute budget. Its actual rate-limit protection is bounded concurrency through `asyncio.Semaphore(max_concurrency)` plus retry with exponential jitter for provider-level transient errors. The default highest application burst is 10 concurrent nutrition lookups, while the explicit test uses a limit of 2 and verifies the bound.

If a provider turns HTTP 429 into `ProviderError`, `run_with_retries` retries it because `ProviderError` is in the retry set. This strategy has not been verified against a real observed 429 in this report; it is covered structurally by tests around provider failure and retry behavior rather than live rate-limit testing.

4. Robustness and Error Handling

4.1 Wrapping the AI module

`AIService` wraps `ai.identify_ingredients`, `ai.NutritionProvider.lookup`, and `ai.compute_totals`. The first two external/provider calls are executed through `run_with_retries`, which uses tenacity's retry loop with `AsyncRetrying`, `stop_after_attempt(settings.retry_attempts)`, and exponential jitter capped at 2.0 seconds. It retries `ProviderError`, `TimeoutError`, and `OSError`; it re-raises after attempts are exhausted.

The wrapper logs start, cache hits, cache misses, failures, and completion counts through Python's `logging` module. It does not currently add a custom per-call timeout; timeout exceptions are handled if raised by lower layers. Input validation happens before provider calls for image file type and size, and pydantic validation guards provider output from the provided AI module.

4.2 Failure-mode analysis

One concrete exercised failure mode is a nutrition provider failure for one ingredient. The test `test_pipeline_handles_one_failed_lookup` creates two ingredients, makes the service raise `ProviderError` for one of them, and then calls `lookup_nutrition_parallel`. The system returns facts for the successful ingredient and one warning instead of crashing the whole analysis.

The user-facing result for this class of failure is a partial or failed structured analysis, depending on whether at least one nutrition lookup succeeds. The logs include `nutrition_lookup_failed` with the

ingredient name. A related test, `test_pipeline_redacts_api_key_from_provider_errors`, verifies that provider error messages containing `api_key=` are sanitized before becoming warnings.

4.3 Input validation

Entry point	Validation and user-visible behavior
CLI image path	<code>validate_image_path</code> checks that the file exists, is under the configured byte limit, and has PNG/JPEG magic bytes. The CLI prints <code>Invalid image: ...</code> and exits with code 2.
HTTP upload	<code>save_upload_bytes</code> rejects empty, oversized, non-PNG, and non-JPEG uploads. The API returns HTTP 422 with a clear detail string.
Filesystem upload name	User filenames are not trusted. The app generates a UUID-prefixed safe name, preserves only a safe suffix, and checks the resolved path stays inside <code>UPLOAD_DIR</code> .
Environment	<code>Settings</code> validates numeric bounds and normalizes <code>LOG_LEVEL</code> ; invalid log levels raise a pydantic validation error.
API history lookup	Missing analysis ids return HTTP 404 with <code>analysis not found</code> .

5. Storage and Caching

5.1 Schema

The persistent storage is PostgreSQL through `asyncpg`. Analysis records are stored as JSONB payloads in one table; this keeps the pydantic response model as the source of truth for detailed ingredients and totals.

```
create table if not exists analyses (
    id text primary key,
    created_at timestamptz not null,
    image_path text not null,
    status text not null,
    payload jsonb not null
);

create table if not exists nutrition_cache (
    key text primary key,
    ingredient_name text not null,
    payload jsonb not null,
    expires_at timestamptz not null,
    updated_at timestamptz not null
);
```

The cached derived fields are the nutrition facts returned by the nutrition provider. The source-of-truth for final analysis output is the serialized `AnalysisRecord` payload in `analyses`. The schema does not split ingredients into a separate relational table, so querying individual ingredients across all analyses would require JSONB queries or a future migration.

5.2 Caching policy

The app caches nutrition facts by normalized ingredient name. Normalization lowercases the string, strips non-alphanumeric separators, and collapses whitespace, so `" White Rice (Cooked) "` becomes `"white rice cooked"`. The default TTL is 86400 seconds, or 24 hours.

Offline mode uses `InMemoryNutritionCache`; online mode uses `PostgresNutritionCache`. Cache entries expire by timestamp, and expired PostgreSQL rows are ignored and deleted on lookup for that key. This report does not claim a measured cache hit rate from a production-like demo run; tests cover cache hit, miss, expiry, clear, schema creation, and unavailable database handling.

6. Testing

6.1 Strategy and coverage

Tests are written with `pytest` and are designed to run offline. The AI module is mocked with fake VLM and nutrition providers in `tests/conftest.py`; API tests use FastAPI's `TestClient`; OpenRouter tests use fake client objects rather than live network access. The required provided smoke tests remain enabled.

The verified commands were run from the project virtual environment on Windows/Python 3.12.10:

```
.venv\Scripts\python.exe -m pytest tests\test_ai_smoke.py -v -p no:cacheprovider
.venv\Scripts\python.exe -m pytest --cov=foodanalyzer --cov-fail-under=60 --cov-report=term-missing -p no:cacheprovider
```

Suite	Result
Provided <code>tests/test_ai_smoke.py</code>	29 passed
Full offline suite	88 passed
<code>foodanalyzer/</code> coverage	85.62%

The lowest covered area is the PostgreSQL repository implementation because most tests use in-memory storage for speed and offline reproducibility. The unused `RateLimiter` helper is also uncovered because the actual concurrency path currently uses a direct semaphore in `concurrency.pipeline`.

6.2 Notable tests

- `test_api_analyze_offline` is the API happy path: it uploads `data/rice_chicken_broccoli.png`, receives HTTP 200, and verifies a completed analysis with positive calories.
- `test_pipeline_bounds_parallelism` is the concurrency test: it tracks active lookup calls and asserts the observed maximum stays within the configured semaphore limit.
- `test_openrouter_vlm_retries_non_json_with_stricter_prompt` exercises malformed model output and verifies the adapter retries with stricter JSON instructions.
- `test_api_rejects_non_image` and `test_validate_image_path_rejects_non_image` verify invalid image handling before provider calls.

The most useful missing test would be an online-mode integration test against a disposable PostgreSQL service plus mocked USDA/VLM HTTP behavior, because it would cover the real `asyncpg` repository path without depending on live providers.

7. Deployment and Reproducibility

7.1 Docker image

The Dockerfile is a single-stage image based on `python:3.11-slim`. It installs pinned dependencies from `requirements.txt`, copies the repository, sets `OFFLINE_MODE=true`, exposes port 8000, and starts:

```
uvicorn foodanalyzer.api:app --host 0.0.0.0 --port 8000
```

The documented build and run commands are:

```
docker build -t foodanalyzer .
docker run --env OFFLINE_MODE=true -p 8000:8000 foodanalyzer
docker compose up --build app
```


The Docker image size was not measured for this report. The repository also includes `docker-compose.yml` with a PostgreSQL 16 service and an app service that connects to `postgresql://postgres:dev@db:5432/foodanalyzer`.

7.2 Configuration

The app reads configuration through `foodanalyzer.config.Settings`, with `.env` support from `pydantic-settings`. Important variables are:

- `APP_NAME`: FastAPI title; defaults to `AI Food Analyzer`.
- `OFFLINE_MODE`: if true, uses fake providers and in-memory storage/cache.
- `LOG_LEVEL`: defaults to `INFO`; invalid values are rejected.
- `DATABASE_URL`: required for online PostgreSQL history and nutrition cache.
- `NUTRITION_CACHE_TTL_SECONDS`: defaults to 86400.
- `MAX_IMAGE_SIZE_MB`: defaults to 5.
- `MAX_CONCURRENCY`: defaults to 10.
- `RETRY_ATTEMPTS`: defaults to 3.
- `HTTP_PORT`: defaults to 8000.
- `UPLOAD_DIR`: defaults to `uploads`.
- `LLM_PROVIDER`, `LLM_MODEL`, and provider API keys: required for online VLM calls.
- `OPENROUTER_*`: required only when using the OpenRouter adapter.
- `NUTRITION_PROVIDER`: defaults to `usda`; only USDA is implemented in the app layer.
- `USDA_API_KEY`: required for online USDA lookup.

8. Limitations and Future Work

- The offline benchmark does not demonstrate real speedup because the fake provider has almost no latency. A better benchmark would add controlled artificial delay or mock HTTP latency.
- There is no multi-provider failover. If the configured VLM provider fails after retries, the analysis fails.
- There is no token-aware or request-per-minute rate limiter; the current protection is bounded concurrency plus retry.
- The PostgreSQL analysis schema stores detailed results as JSONB payloads, which is simple but less convenient for relational analytics over ingredients.
- Online behavior depends on external VLM and USDA availability, correct keys, model image support, and provider response quality.

9. Tools and Acknowledgements

The course-provided `ai/` package supplies the core AI operations: VLM ingredient identification, USDA nutrition lookup, pydantic schemas, and totals calculation. The team built the surrounding software-engineering layer: configuration, CLI, HTTP API, HTMX UI, storage, cache, retries, validation, Docker, and tests.

AI coding assistants were used as engineering collaborators for implementation support, debugging, README preparation, and report drafting. The generated suggestions were reviewed against the repository and adjusted to match the actual code and assignment rules. The final report intentionally avoids claims that were not verified in the current app.

References

- [1] AI Academy, National AI Center. *AI-ENG-110 Software Engineering Final Project Brief*, v2.0, May 13, 2026.
- [2] Course Topic 2 specification: TOPIC.md, AI Food Analyzer.
- [3] FastAPI documentation: <https://fastapi.tiangolo.com/>
- [4] Pydantic Settings documentation: https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] Tenacity retrying library: <https://tenacity.readthedocs.io/>
- [6] USDA FoodData Central API: <https://fdc.nal.usda.gov/api-guide.html>
- [7] OpenRouter API documentation: <https://openrouter.ai/docs>

A. Appendix – Reproducing the benchmark

```
git clone https://github.com/GeminusF/ai-eng-110-final-project-ss-stomachslave
cd ai-eng-110-final-project-ss-stomachslave
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe scripts\bench.py --image data\rice_chicken_broccoli.png
```

Observed output used in this report:

```
mode,ingredients,seconds
sequential,3,0.0013
concurrent,3,0.0013
```

End of report.